

Guia de testes da API de Produtos

Backend Node.js com Express, Jest e SQLite.

1. Como executar

```
npm test
```

```
npm test -- --runInBand
```

O primeiro comando executa todas as suites. O segundo executa uma suite por vez.

2. Organizacao e objetivo

- src/models/Produto.test.js: testa validar com dados reais, sem mocks.

- src/services/ProdutoService.test.js: testa regras de negocio isolando dependencias.

- src/controllers/ProdutoController.test.js: testa respostas HTTP e delegacao ao servico.

Um teste unitario exercita uma unidade pequena e substitui dependencias externas.

3. Testes do modelo

A funcao validar aceita produto com nome, categoria permitida e preco numerico maior que zero.

Casos: produto correto; nome vazio; preco negativo; categoria fora da lista.

Exemplo

```
test('rejeita preco invalido', () => {  
  expect(validar({ nome: 'Racao', categoria: 'alimentacao', preco: -10 }))  
    .toBe(false);  
});
```

4. Testes do servico

ProdutoService chama o repositorio, valida dados e lanca HttpError quando necessario.

O repositorio e o modelo sao mocks porque SQLite nao e responsabilidade dessa camada.

Casos: listagem com filtro, busca encontrada, 404, criacao valida/invalida,

atualizacao valida/inexistente e exclusao de produto existente.

Guia de testes da API de Produtos

5. Testes do controller

O controller recebe req e res simples. Não é preciso iniciar o Express no teste unitário. criarResposta cria mocks para status, json, send e set.

Casos: listagem incrementa a sessão e retorna 200; criação retorna 201;

erros seguem para next(erro); exportação define text/csv e envia o conteúdo CSV.

6. Recursos do Jest usados

test(nome, funcao): declara um caso independente.

expect(valor): inicia uma verificação.

toBe(valor): compara primitivos ou a mesma referência.

toEqual(valor): compara conteúdo de objetos e arrays.

toThrow(texto ou classe): confirma que uma função lança erro.

jest.mock(caminho, factory): substitui módulo importado por uma versão controlada.

jest.fn(): cria função simulada e registra suas chamadas.

mockReturnValue(valor): define o retorno do mock.

mockImplementation(funcao): define comportamento, por exemplo lançar erro.

mockReturnThis(): permite res.status(...).json(...) no mock de resposta.

toHaveBeenCalledWith(...): confere argumentos de uma chamada.

not.toHaveBeenCalled(): confirma que uma chamada proibida não ocorreu.

jest.clearAllMocks(): limpa o histórico dos mocks antes de cada teste.

7. Fluxo para criar um novo teste

1. Escolha uma regra: exemplo, produto inexistente retorna 404.
2. Faça mock apenas das dependências da camada.
3. Prepare entradas e retornos dos mocks.
4. Execute a função e verifique retorno, status HTTP ou erro.
5. Verifique argumentos e chamadas que não devem ocorrer.

Use Supertest para testes de integração de rota, middleware, sessão e Express.

Os testes atuais são unitários: rápidos, isolados e chamam controller/service diretamente.